

Behavioral Synthesis of ICs

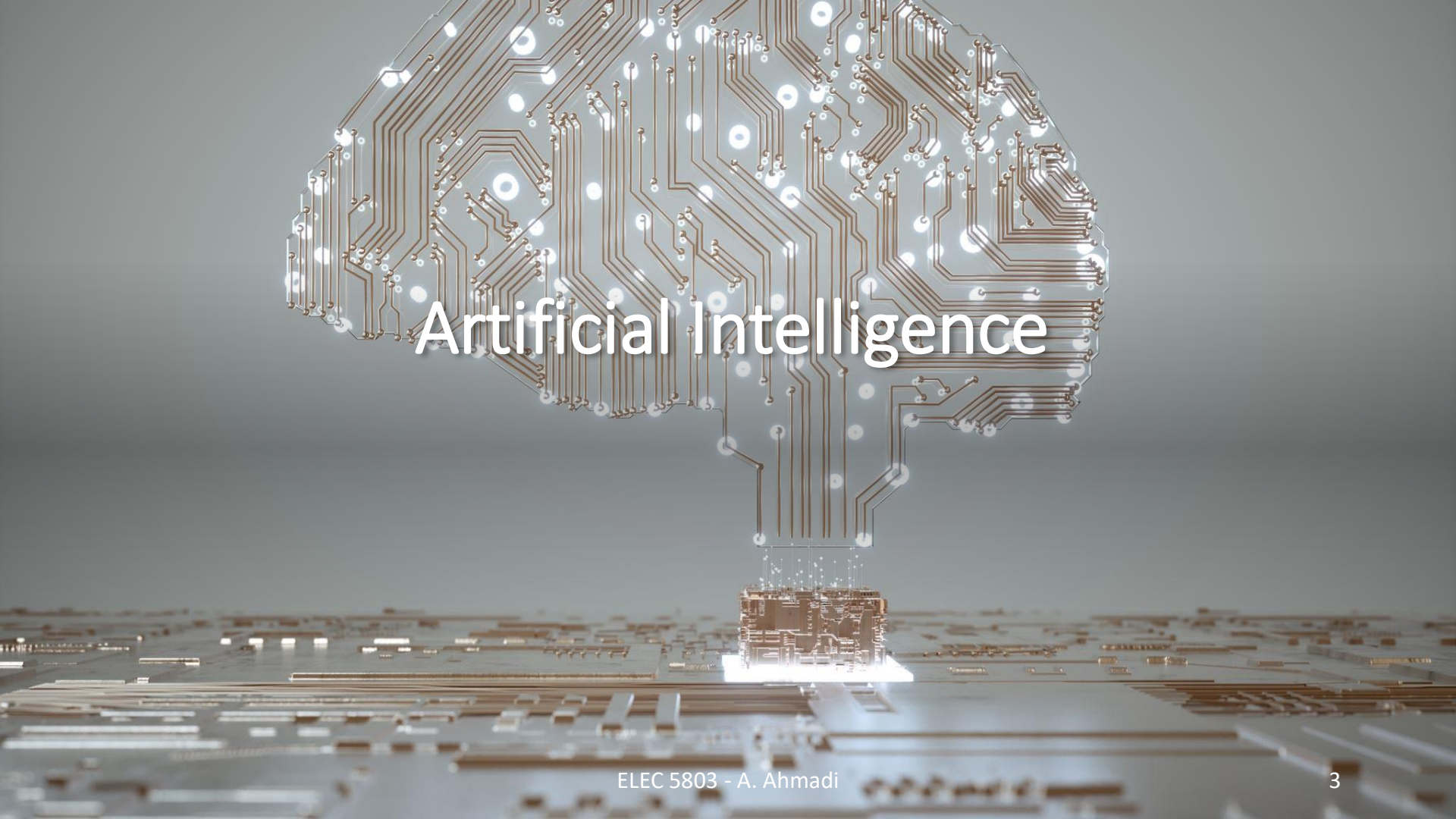
Domain Specific HLS

Credentials:

1. Huhn, Sebastian, and Rolf Drechsler. *Design for Testability, Debug and Reliability*. Springer International Publishing, 2021.
2. Fahim, Farah, et al. "hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices." *arXiv preprint arXiv:2103.05579* (2021).
3. Cong, Jason, Jason Lau, Gai Liu, Stephen Neuendorffer, Peichen Pan, Kees Vissers, and Zhiru Zhang. "FPGA HLS today: successes, challenges, and opportunities." *ACM Transactions on Reconfigurable Technology and Systems (TRETs)* 15, no. 4 (2022): 1-42.
4. Khalid, Ayesha, Goutam Paul, and Anupam Chattopadhyay. *Domain Specific High-Level Synthesis for Cryptographic Workloads*. Springer Singapore, 2019.

Outline

- **What is intelligence?**
 - **What is Artificial Intelligence?**
 - **Acting humanly: The Turing Test approach**
- **Machine Learning**
 - **Types of Learning**
 - **Artificial Neural Networks**
 - **Deep Neural Networks**
- **Design of Hardware for Processing DNNs**
 - **Key Metrics and Design Objectives**
 - **Design Steps**
 - **AI Hardware Accelerators**

The image features a large, stylized brain shape in the upper half, constructed from a complex network of brown circuit lines and glowing white circular nodes. Below the brain, a small, detailed microchip sits on a white rectangular base, with faint blue light emanating from it. The background is a blurred, high-angle view of a circuit board. The text "Artificial Intelligence" is centered over the brain.

Artificial Intelligence

What is intelligence?

- We call ourselves **Homo Sapiens** —man the wise— because our intelligence is so important to us!
- For thousands of years, we have tried to understand how we think; that is, how a mere handful of matter can **perceive**, **understand**, **predict**, and **manipulate** a world far larger and more complicated than itself.
- Generally, intelligence can be described as the ability to **perceive** or infer **information**, and to retain it as **knowledge** to be applied towards **adaptive behaviors** within an **environment** or context. Intelligence has been defined in many ways:
 - the capacity for logic,
 - understanding,
 - self-awareness,
 - learning,
 - emotional knowledge,
 - reasoning,
 - planning,
 - creativity,
 - critical thinking,
 - problem-solving.

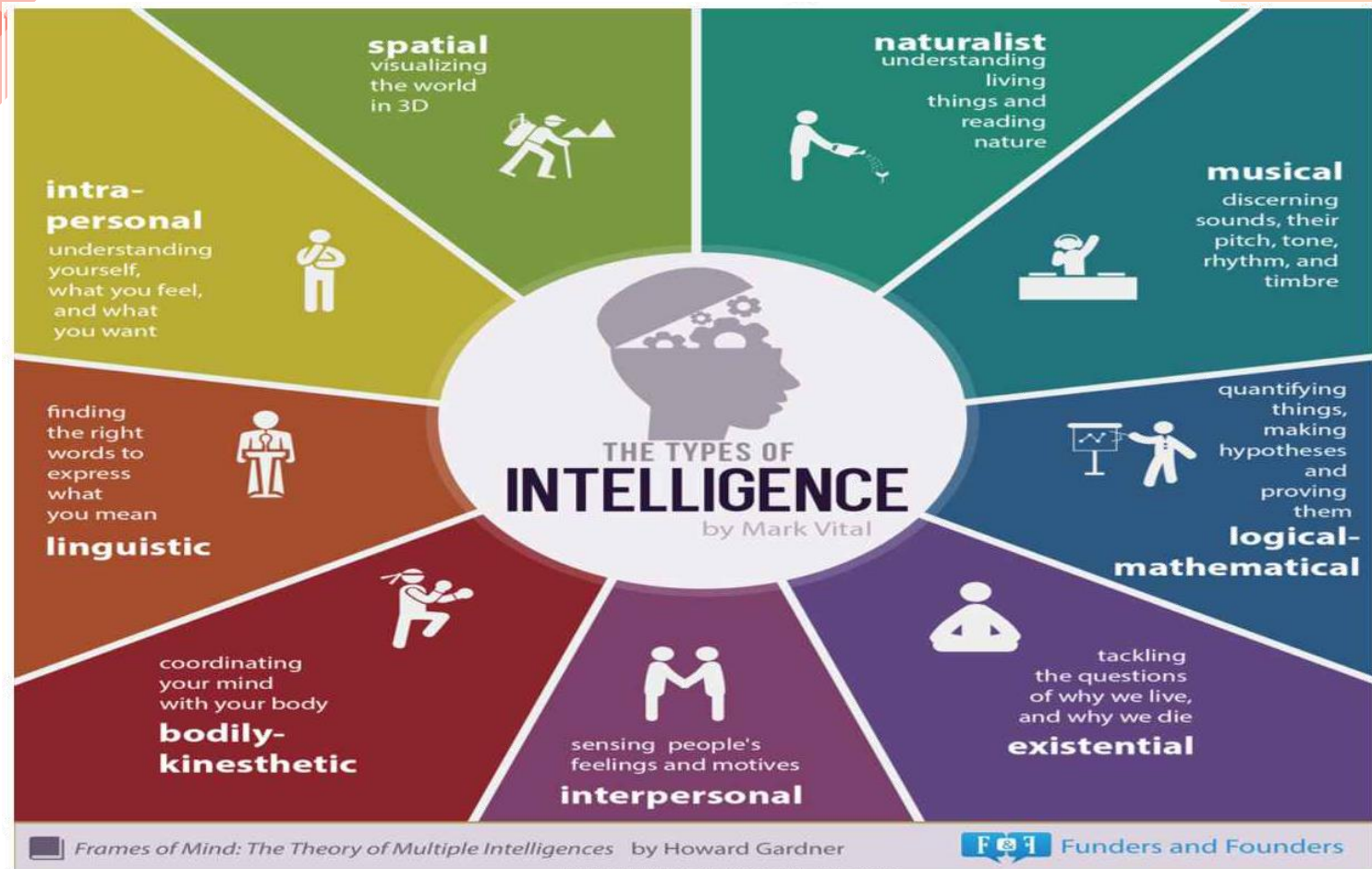
What is intelligence?

- Several subcategories of intelligence, such as **emotional intelligence** or **social intelligence**, are heavily debated as to whether they are traditional forms of intelligence.
- They are generally thought to be distinct processes that occur, though there is speculation that they tie into traditional intelligence more than previously suspected.
- Despite their use of the word 'Intelligence,' some terms may have little or nothing to do with the mentioned **cognitive processes**.
- **Gardner's theory** of **Multiple Intelligences** suggests that traditional psychometric views of intelligence are too limited.
- Gardner first outlined his theory in his 1983 book *Frames of Mind: The Theory of Multiple Intelligences*, where he suggested that all people have **different kinds of intelligences**.
- Gardner proposed that there are eight intelligences and has suggested the possible addition of a ninth known as "existentialist intelligence."
- Critics argue that Gardner's definition of intelligence is too broad and that his eight different "intelligences" simply represent talents, personality traits, and abilities.

What is intelligence?

→ Howard Gardener described 9 types of intelligence [1]:

- Naturalist (nature smart)
- Musical (sound smart)
- Logical-Mathematical (number/reasoning smart)
- Existential (life smart)
- Interpersonal (people smart)
- Bodily-Kinesthetic (body smart)
- Linguistic (word smart)
- Intra-Personal (self smart)
- Spatial (picture smart)



What is Artificial Intelligence?

- AI is the science and engineering of creating intelligent machines that have the ability to achieve goals like humans do, according to John McCarthy, the computer scientist who coined the term in the 1950s.
- AI currently encompasses a huge variety of subfields, ranging from the general (learning and perception) to the specific, such as playing chess, proving mathematical theorems, writing poetry, driving a car on a crowded street, and diagnosing diseases.
- AI is relevant to any intellectual task; it is truly a universal field. In our broad definition, AI is a collective term for computer systems that can **sense their environment**, **think**, **learn**, and **take action** in response to what they're sensing and their objectives.
- AI currently encompasses a huge variety of subfields, ranging from the general (learning and perception) to the specific, such as playing chess, proving mathematical theorems, writing poetry, driving a car on a crowded street, and diagnosing diseases.
- AI is relevant to any intellectual task; it is truly a universal field.

Humanly

Rational

Figure 1.1 Some definitions of artificial intelligence, organized into four categories.

Thinking

thought processes
and reasoning

Thinking Humanly

“The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense.” (Haugeland, 1985)

“[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . .” (Bellman, 1978)

Thinking Rationally

“The study of mental faculties through the use of computational models.” (Charniak and McDermott, 1985)

“The study of the computations that make it possible to perceive, reason, and act.” (Winston, 1992)

Behavior

action processes
and reaction

Acting Humanly

“The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990)

“The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)

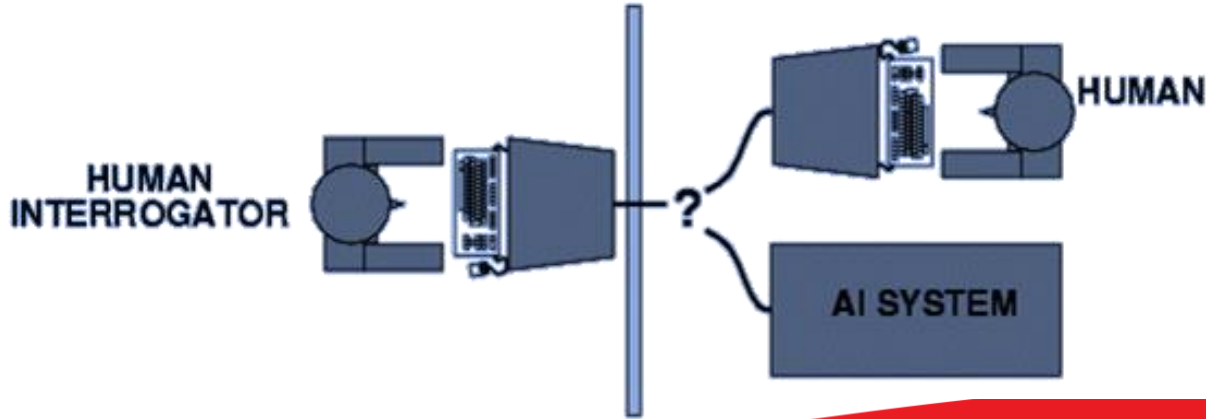
Acting Rationally

“Computational Intelligence is the study of the design of intelligent agents.” (Poole *et al.*, 1998)

“AI . . . is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)

Acting humanly: The Turing Test approach

- The **Turing Test**, proposed by Alan Turing (1950), was designed to provide a satisfactory operational definition of intelligence.
- A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.
- Turing's test deliberately avoided direct physical interaction between the interrogator and the computer, because physical simulation of a person is unnecessary for intelligence.





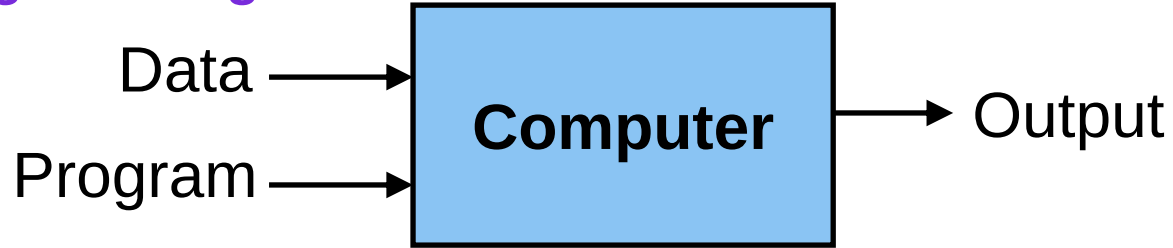
Machine Learning

Machine Learning

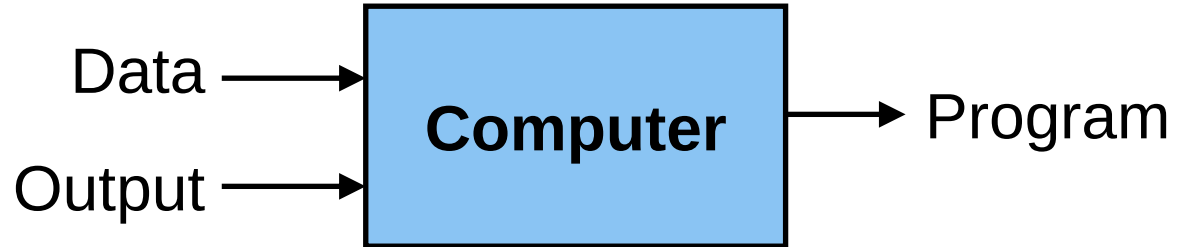
- Within AI is a large sub-field called **machine learning**, which was defined in 1959 by Arthur Samuel as “the field of study that gives computers the ability to learn without being explicitly programmed.”
- That means a single program, once created, will be able to learn how to do some intelligent activities outside the notion of programming. This contrasts with purpose-built programs whose behavior is defined by hand-crafted heuristics that explicitly and statically define their behavior.
- The advantage of an effective machine learning algorithm is clear. Instead of the laborious and hit-or-miss approach of creating a distinct, custom program to solve each individual problem in a domain, a single machine learning algorithm simply needs to learn, via a process called training, to handle each new problem.

Machine Learning

Traditional Programming



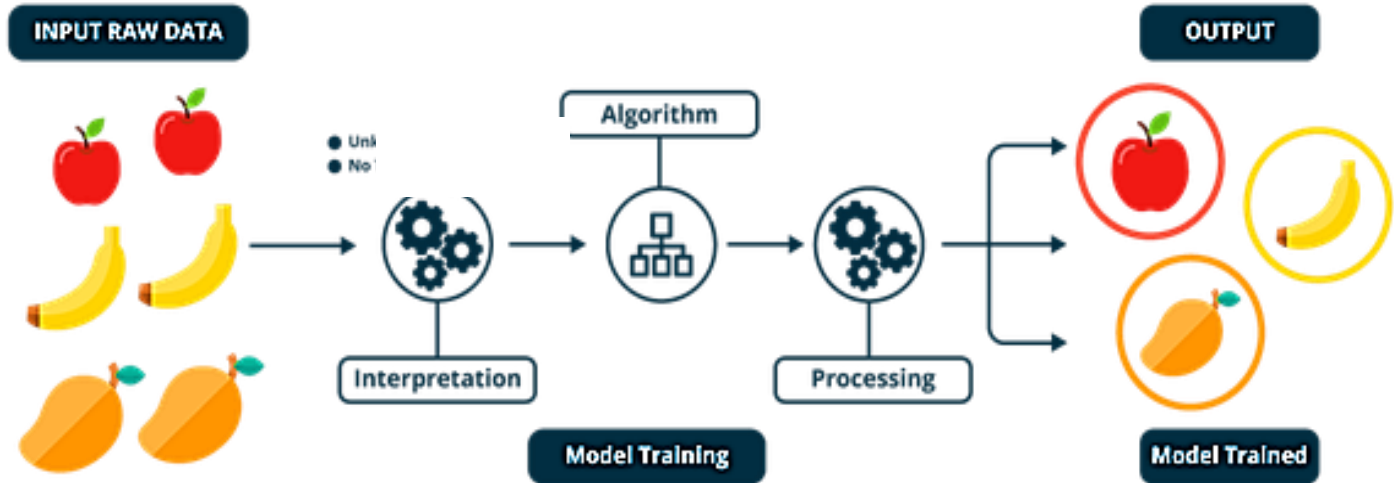
Machine Learning



Machine Learning

- Technically, ML is a field of artificial intelligence that involves the design and implementation of algorithms for computers to evolve their behavior from observations, examples, images, sensors, data, experience, This might mean:
 - Automating automation
 - Getting computers to program themselves
 - Writing software is the bottleneck
 - Let the data do the work instead!
- An agent is learning if it improves its performance on future tasks after making observations about the world.
- Learning can range from the trivial, as exhibited by jotting down a phone number, to the profound, as exhibited by Albert Einstein, who inferred a new theory of the universe.

Machine Learning



Types of Learning

- There are three types of feedback that determine the three main types of machine learning algorithms:
 - **Supervised** (inductive) learning
 - Training data includes desired outputs
 - **Unsupervised** learning
 - Training data does not include desired outputs
 - **Reinforcement** learning
 - Rewards from sequence of actions
 - **Semi-supervised** learning
 - Training data includes a few desired outputs
- There are tens of thousands of machine learning algorithms, and hundreds new every year! In a nutshell, every machine learning algorithm has three major components:
 - **Representation**
 - **Evaluation**
 - **Optimization**

Machine Learning

→ There are variety of ML representations:

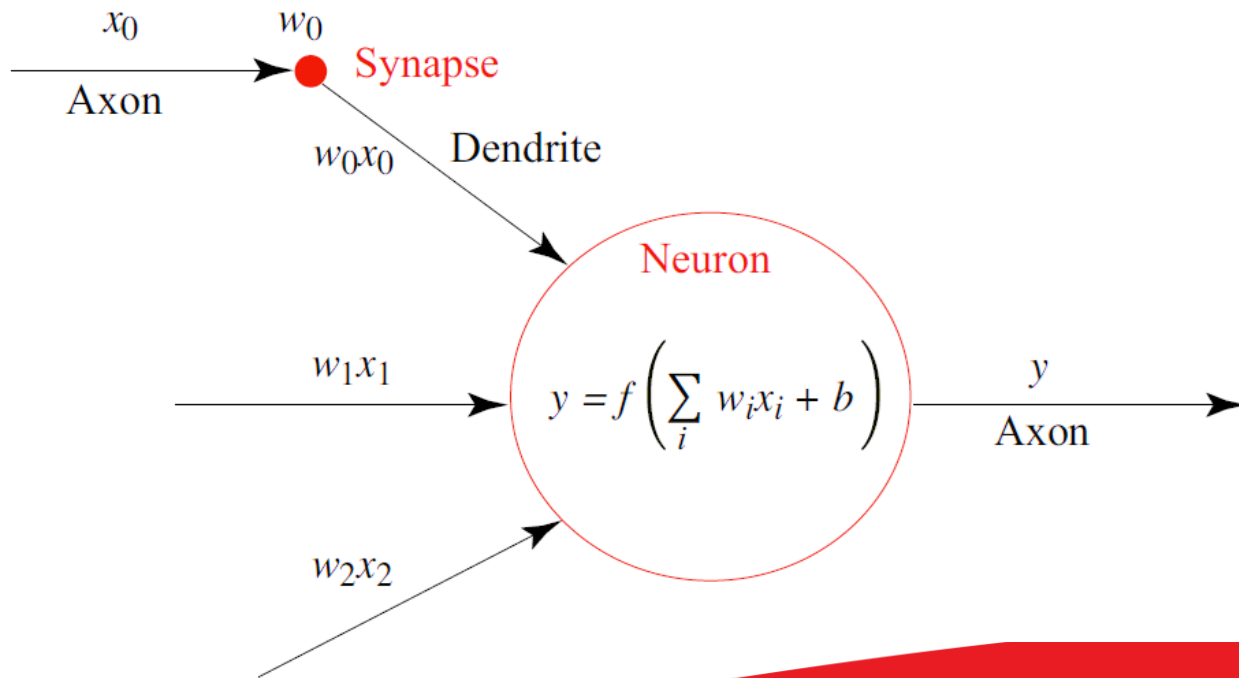
- Neural networks
- Decision trees
- Graphical models (Bayes/Markov nets)
- Support vector machines
- Sets of rules / Logic programs
- Instances
- Model ensembles
- etc.

Artificial Neural Networks

- Within the **machine learning** field, there is an area that is often referred to as **brain-inspired computation**.
- Since the brain is currently the best “machine” we know of for learning and solving problems, it is a natural place to look for inspiration. Therefore, a brain-inspired computation is a program or algorithm that takes some aspects of its basic form or functionality from the way the brain works.
- This is in contrast to attempts to create a brain, but rather the program aims to emulate some aspects of how we understand the brain to operate. Although scientists are still exploring the details of how the brain works, it is generally believed that the main computational element of the brain is the neuron.
- There are approximately 86 billion neurons in the average human brain. The neurons themselves are connected by a number of elements entering them, called dendrites, and an element leaving them, called an axon.

Neural Networks

→ **Figure 4:** Connections to a neuron in the brain. x_i , w_i , $f(\cdot)$, and b are the activations, weights, nonlinear function, and bias, respectively.

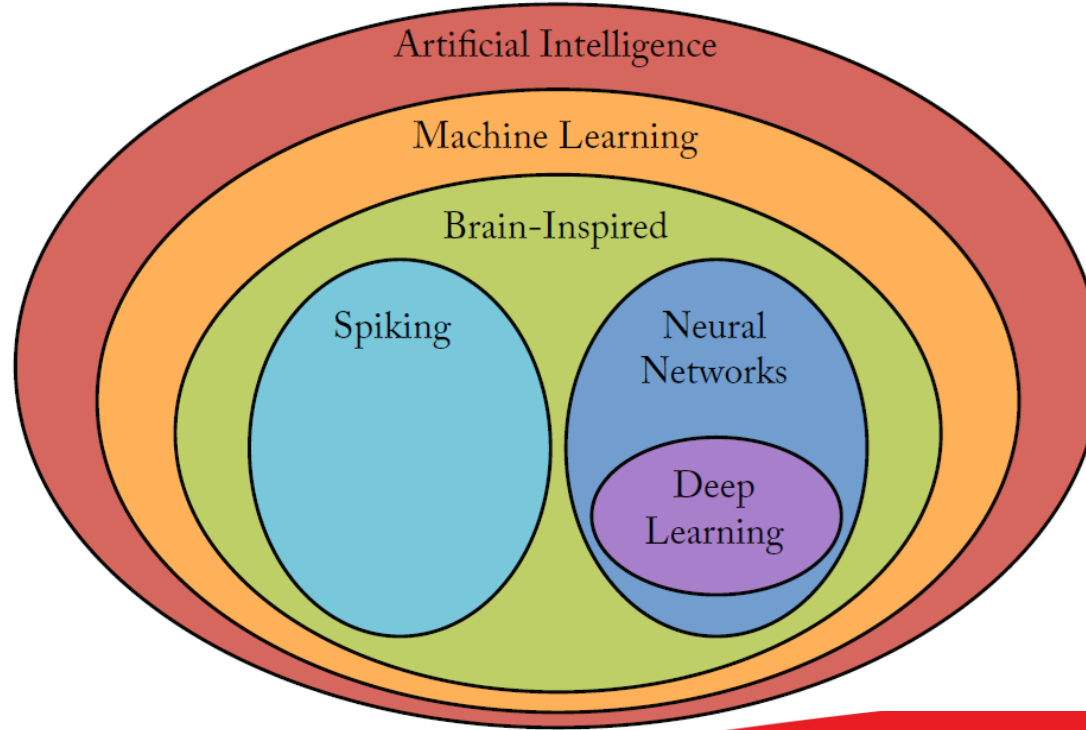


Neural Networks

- Within the brain-inspired computing paradigm, there is a subarea called **spiking neural network**.
- In this subarea, inspiration is taken from the fact that the communication on the dendrites and axons are spike-like pulses and that the information being conveyed is not just based on a spike's amplitude.
- Instead, it also depends on the time the pulse arrives and that the computation that happens in the neuron is a function of not just a single value but the width of pulse and the timing relationship between different pulses.
- The IBM TrueNorth and IBM NorthPole, and Intel's Loihi projects are examples of work that was inspired by the spiking of the brain.

Artificial Intelligence and Deep Neural Networks

→ The relationship of deep learning to the whole of AI is illustrated in Figure 1.



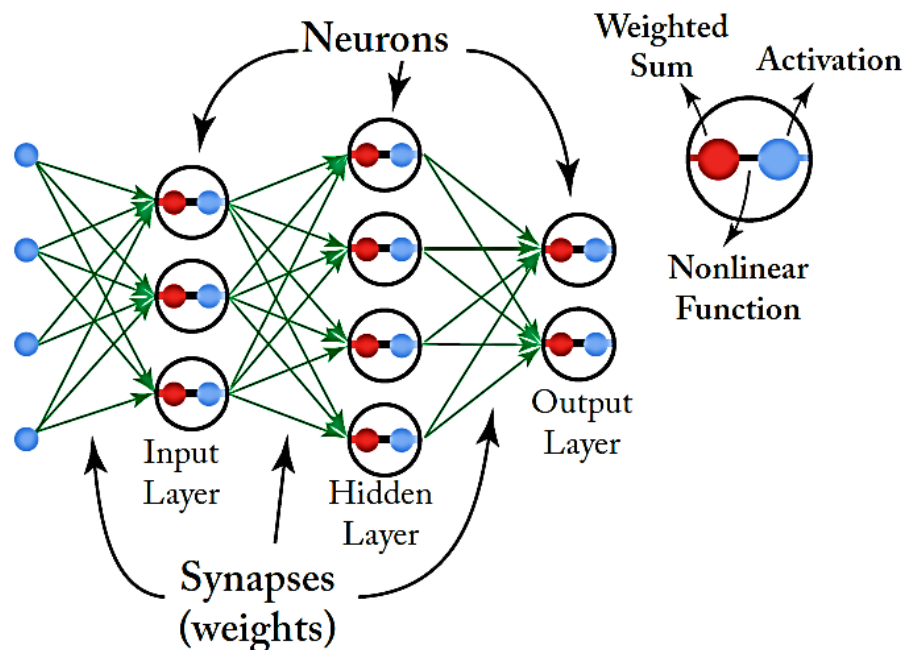
Artificial Neural Networks

- Neural networks take their inspiration from the notion that a neuron's computation involves a **weighted sum of the input values**.
- These **weighted sums** correspond to the value scaling performed by the **synapses** and the combining of those values in the **neuron**.
- The neuron does not directly output that weighted sum because the expressive power of the cascade of neurons involving only linear operations is just equal to that of a single neuron, which is very limited. Instead, there is a functional operation within the neuron that is performed on the combined inputs.
- This operation appears to be a **nonlinear function** that causes a neuron to generate an output only if its combined inputs cross some threshold.
- By analogy, neural networks apply a nonlinear function to the weighted sum of the input values. These nonlinear functions are inspired by biological functions but in ANNs are not meant to emulate the brain.

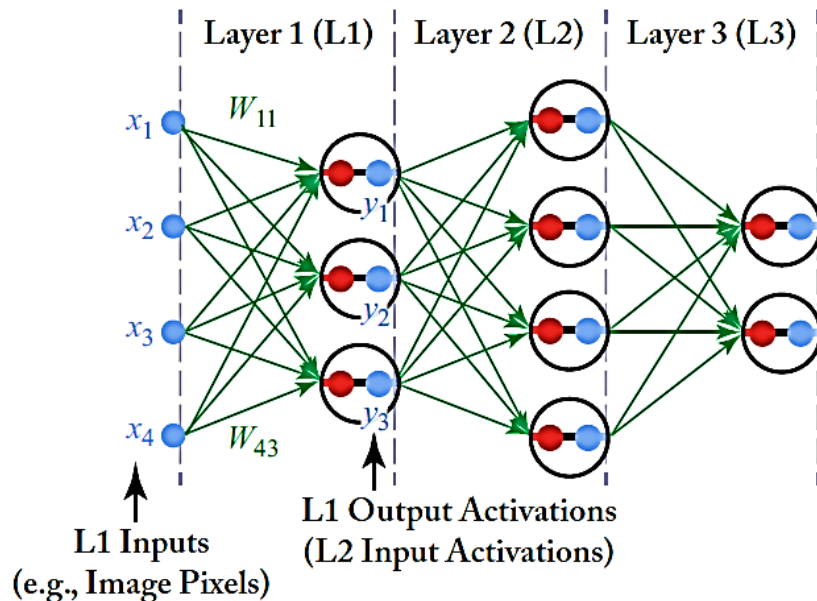
Artificial Neural Networks

- Figure 5a shows a diagram of a three-layer (non-biological) neural network.
- The neurons in the input layer receive some values, compute their weighted sums followed by the nonlinear function, and propagate the outputs to the neurons in the middle layer of the network, which is also frequently called a “hidden layer.”
- A neural network can have more than one hidden layer, and the outputs from the hidden layers ultimately propagate to the output layer, which computes the final outputs of the network to the user.
- To align brain-inspired terminology with neural networks, the outputs of the neurons are often referred to as activations (action potential in biological neurons), and the synapses are often referred to as weights, as shown in Figure 5a.

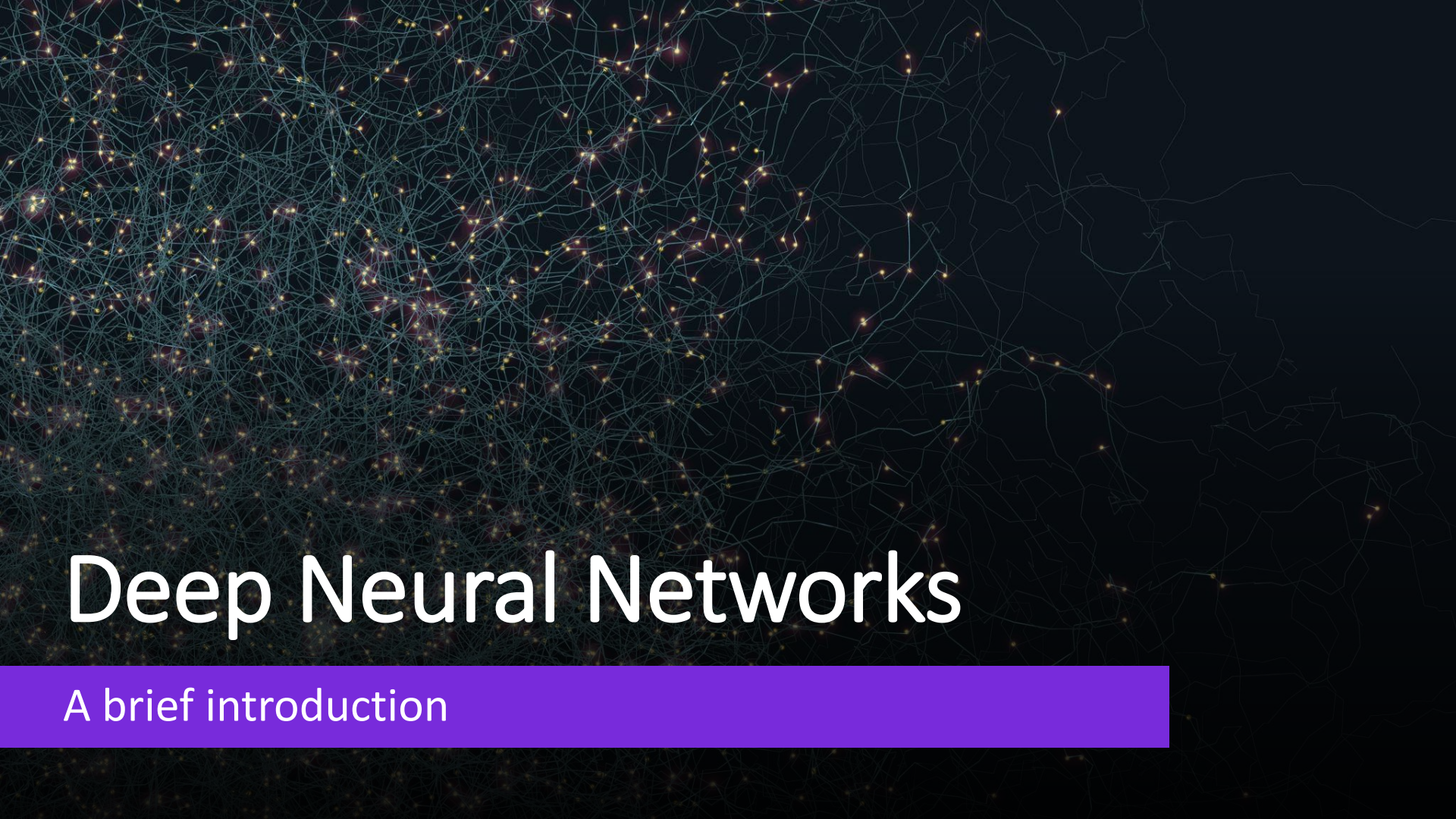
Figure 5: Simple neural network example and terminology.



(a) Neurons and synapses



(b) Compute weighted sum for each layer



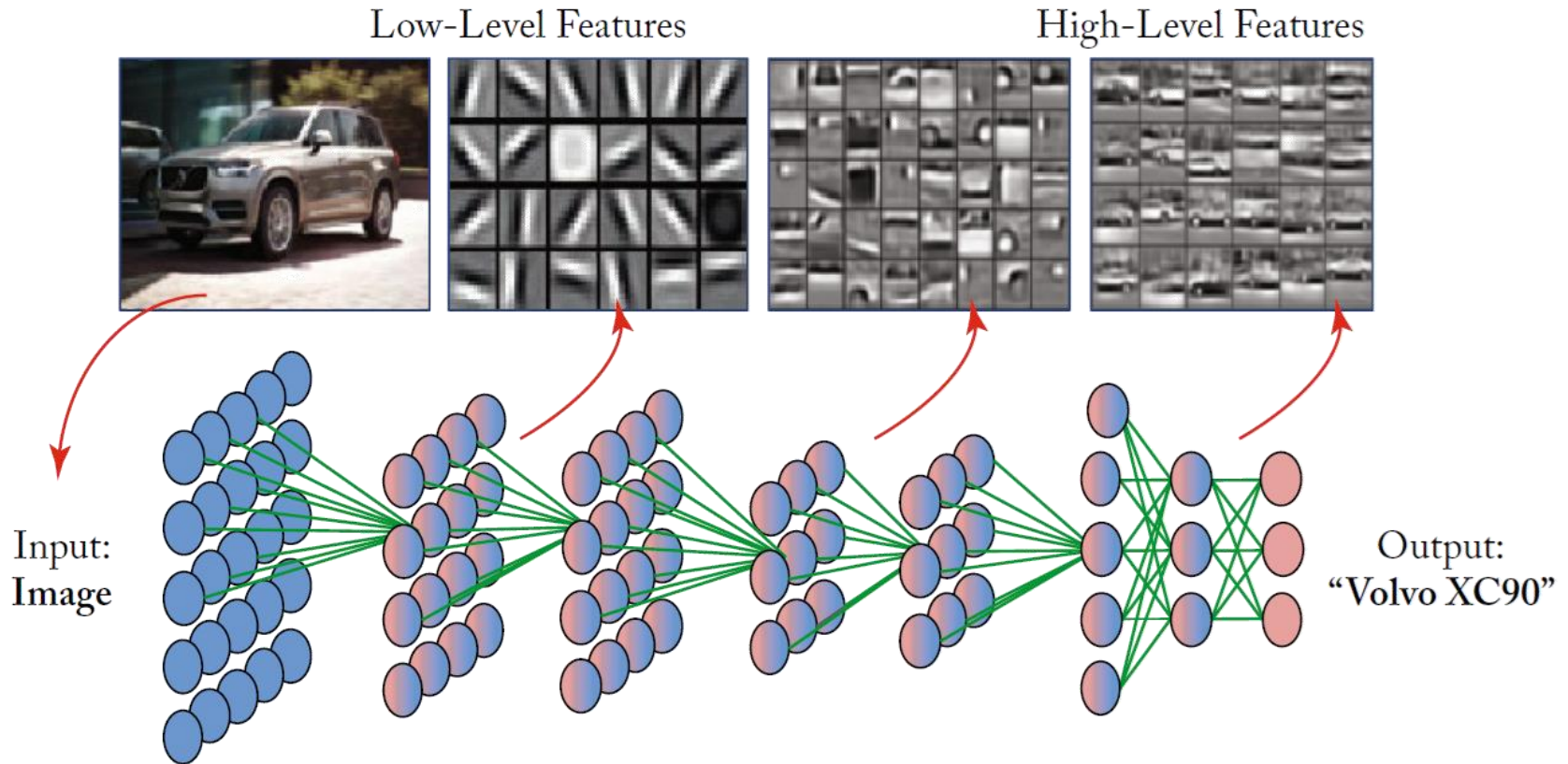
Deep Neural Networks

A brief introduction

Deep Neural Networks

- Within the domain of neural networks, there is an area called **deep learning**, in which the neural networks have more than three layers, i.e., more than one hidden layer.
- Today, the typical numbers of network layers used in deep learning range from 5 to more than a 1,000.
- Here, we will generally use the terminology **deep neural networks (DNNs)** to refer to the neural networks used in deep learning.
- DNNs are capable of learning high-level features with more complexity and abstraction than shallower neural networks.
- An **example** that demonstrates this point is using DNNs to process visual data, as shown in Figure 6.

Figure 6: Example of image classification using deep neural networks. Note that the features go from low level to high level as we go deeper into the network.



Deep Neural Networks

- In this example, pixels of an image are fed into the first layer of a DNN, and the outputs of that layer can be interpreted as representing the presence of different low-level features in the image, such as lines and edges.
- In subsequent layers, these features are then combined into a measure of the likely presence of higher-level features, e.g., lines are combined into shapes, which are further combined into sets of shapes.
- Finally, given all this information, the network provides a probability that these high-level features comprise a particular object or scene.
- This deep feature hierarchy enables DNNs to achieve superior performance in many tasks.

Deep Neural Networks

- Since DNNs are an instance of machine learning algorithms, the basic program does not change as it learns to perform its given tasks.
- In the specific case of DNNs, this learning involves **determining the value of the weights** (and biases) in the network and is referred to as **training the network**.
- Once trained, the program can perform its task by computing the output of the network using the weights determined during the training process. Running the program with these weights is referred to as inference.
- When one performs inference using the DNN in Figure 6, the input is image, and the output is a vector of values representing the class probabilities.

Deep Neural Networks

- There is one value for each object class, and the class with the highest value indicates the most likely (predicted) class of object in the image.
- The overarching goal for training a DNN is to determine the weights that maximize the probability of the correct class and minimize the probabilities of the incorrect classes.
- The correct class is generally known, as it is often defined in the training set.
- The gap between the ideal correct probabilities and the probabilities computed by the DNN based on its current weights is referred to as the loss (L).
- Thus, the goal of training DNNs is to find a set of weights to minimize the average loss over a large training set.

Deep Neural Networks

- When **training a network**, the **weights** (ω_{ij}) are usually updated using a **hill-climbing (hill-descending)** optimization process called **gradient descent**.
- In **gradient descent**, a weight is updated by a scaled version of the **partial derivative** of the loss with respect to the weight (i.e., updated to $\omega_{ij}^{t+1} = \omega_{ij}^t - \alpha \frac{\partial L}{\partial \omega_{ij}}$, where α is called the learning rate).
- Note that this **gradient indicates** how the weights should change in order to reduce the loss. The process is repeated iteratively to reduce the overall loss.
- An efficient way to compute the **partial derivatives** of the **gradient** is through a process called **backpropagation**.
- **Backpropagation**, which is a computation derived from the chain rule of calculus, operates by passing values backward through the network to compute how the loss is affected by each weight.

Deep Neural Networks

- This **backpropagation computation** is, in fact, very similar in form to the computation used for inference, as shown in Figure 7.
- Thus, techniques for efficiently performing inference can sometimes be useful for performing **training**. There are, however, some important additional considerations to note.
 - First, **backpropagation** requires **intermediate outputs** of the network to be preserved for the **backward computation**, thus training has increased storage requirements.
 - Second, due to the gradients use for hill-climbing (hill-descending), the **precision requirement for training** is generally higher than inference. Thus, many of the reduced precision techniques are introduced in the research community which are limited to inference only.
- A variety of techniques are used to improve the efficiency and robustness of training.
- For **example**, often, the loss from multiple inputs is computed before a single pass of weight updates is performed. This is called batching, which helps to speed up and stabilize the process

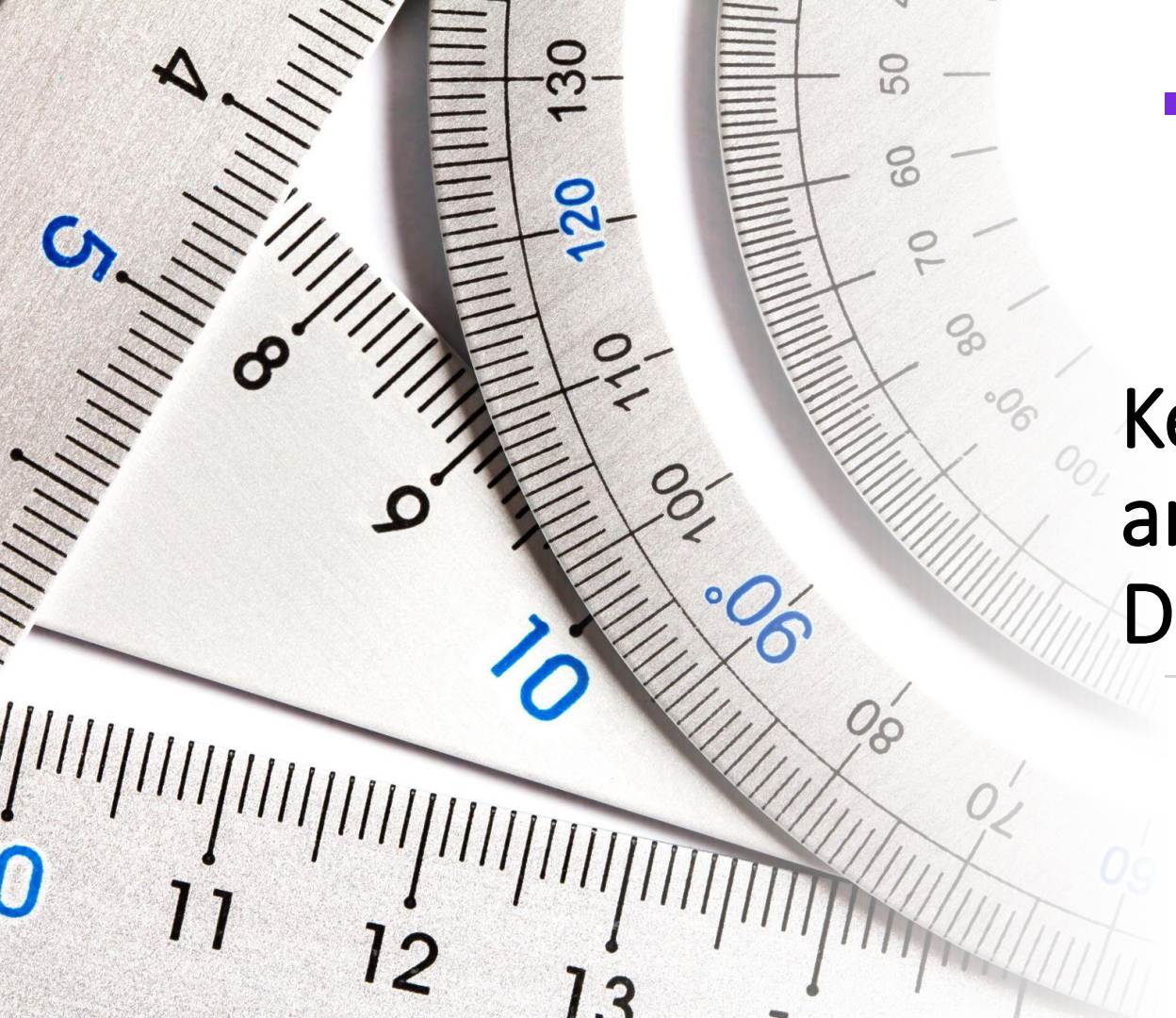
A Concise History of Neural Networks.

→ DNN Timeline:

- 1940s: Neural networks were proposed
- 1960s: Deep neural networks were proposed
- 1989: Neural networks for recognizing hand-written digits (LeNet)
- 1990s: Hardware for shallow neural nets (Intel ETANN)
- 2011: Breakthrough DNN-based speech recognition (Microsoft)
- 2012: DNNs for vision start supplanting hand-crafted approaches (AlexNet)
- 2014+: Rise of DNN accelerator research (Neuflow, DianNao...)

Design of Hardware for Processing DNNs

- Over the past few years, there has been a significant amount of research on efficient processing of DNNs.
- There are key metrics that one should consider when comparing and evaluating the strengths and weaknesses of different designs and proposed techniques and that should be incorporated into design considerations.
- While efficiency is often only associated with the **number of operations per second per Watt** (e.g., floating-point operations per second per Watt as **FLOPS/W** or tera-operations per second per Watt as **TOPS/W**), it is actually composed of many more metrics including **accuracy, throughput, latency, energy consumption, power consumption, cost, flexibility, and scalability**.
- In general, considering a comprehensive set of these metrics is important in order to provide a complete picture of the trade-offs made by a proposed design or technique.



Key Metrics and Design Objectives

Key Metrics and Design Objectives

- **Accuracy** is used to indicate the quality of the result for a given task.
- The fact that DNNs can achieve state-of-the-art accuracy on a wide range of tasks is one of the key reasons driving the popularity and wide use of DNNs today.
- The units used to measure accuracy depend on the task.
- For **instance**, for image classification, accuracy is reported as the **percentage of correctly classified** images, while for object detection, accuracy is reported as the **mean average precision (mAP)**, which is related to the trade off between the **true positives**, **false positives**, and **false negatives**.
- Factors that affect accuracy include the **difficulty of the task** and dataset. For instance, image classification on ImageNet dataset is much more difficult than on MNIST, and **object detection** is usually more difficult than **classification**.
- As a result, a DNN model that performs well on MNIST may not necessarily perform well on ImageNet.

Key Metrics and Design Objectives

- Achieving high **accuracy** on difficult tasks or datasets typically requires more **complex DNN models** (e.g., a larger number of **MAC operations** and more distinct weights, increased diversity in layer shapes, etc.), which can impact how efficiently the hardware can process the DNN model.
- **Accuracy** should therefore be interpreted in the context of the difficulty of the task and dataset.
- Evaluating hardware using well-studied, widely used DNN models, tasks, and datasets can allow one to better interpret the significance of the accuracy metric.

Key Metrics and Design Objectives

- **Throughput** is used to indicate the amount of data that can be processed or the number of executions of a task that can be completed in a given time period. High throughput is often critical to an application.
- For **instance**, processing video at 30 frames per second is often necessary for delivering real-time performance.
- For data analytics, **high throughput** means that more data can be analyzed in a given amount of time.
- As the amount of visual data is growing exponentially, high-throughput big data analytics becomes increasingly important, particularly if an action needs to be taken based on the analysis (e.g., security or attack prevention; medical diagnosis or drug discovery).
- Throughput is often generically reported as the **number of operations per second**. In the case of inference, throughput is reported as **inferences per second**.

Key Metrics and Design Objectives

- **Latency** measures the time between when the input data arrives to a system and when the result is generated.
- **Low latency** is necessary for **real-time interactive applications**, such as augmented reality, autonomous navigation, and robotics.
- Latency is typically reported in **seconds per inference**. Throughput and latency are often assumed to be directly derivable from one another. However, they are actually quite distinct.
- A prime **example** of this is the well-known approach of batching input data (e.g., batching multiple images or frames together for processing) to increase throughput since it amortizes overhead, such as loading the weights; however, batching also increases latency (e.g., at 30 frames per second and a batch of 100 frames, some frames will experience at least 3.3 second delay), which is not acceptable for real-time applications, such as high-speed navigation where it would reduce the time available for course correction.
- Achieving **low latency and high throughput simultaneously** can sometimes be at odds depending on the approach and both should be reported.

Key Metrics and Design Objectives

- When considering a system comprised of multiple **processing elements** (PEs), where a PE corresponds to a simple or primitive core that performs a single **MAC operation**, the **number of operations per second** can be further decomposed as follows:

$$\frac{\text{operations}}{\text{second}} = \underbrace{\left(\frac{1}{\frac{\text{cycles}}{\text{operation}}} \times \frac{\text{cycles}}{\text{second}} \right)}_{\text{for a single PE}} \times \text{number of PEs} \times \text{utilization of PEs}$$

- The first term reflects the peak **throughput of a single PE**, the second term reflects the amount of **parallelism**, while the last term reflects degradation due to the inability of the architecture to effectively **utilize the PEs**.
- Since the main operation for processing DNNs is a MAC, we will use number of operations and number of **MAC operations** interchangeably.

Key Metrics and Design Objectives

- One can increase the **peak throughput of a single PE** by increasing the **number of cycles per second**, which corresponds to a **higher clock frequency** achieved by reducing the **critical path** at the circuit or micro-architectural level; alternatively, one can also **reduce the number of cycles per operations**, which can be affected by the design of the MAC (e.g., a **bit-serial, multi-cycle MAC** would have more cycles per operation).
- While the above approaches increase the **throughput of a single PE**, the overall throughput can be increased by increasing the number of PEs, and thus the maximum **number of MAC operations** that can be **performed in parallel**.
- The **number of PEs** is dictated by the area of the PE and the **area cost of the system**. If the area cost of the system is fixed, then increasing the number of PEs requires either reducing the area per PE or trading off on-chip storage area for more PEs.
- Reducing on-chip storage, however, can affect the utilization of the PEs.

Key Metrics and Design Objectives

- Reducing the **area per PE** can also be achieved by reducing the logic associated with delivering operands to a MAC.
- This can be achieved by controlling multiple MACs with a single piece of logic. This is analogous to the situation in instruction-based systems such as CPUs and GPUs that reduce instruction bookkeeping overhead by using large aggregate instructions (e.g.,
 - **single-instruction multiple-data (SIMD)**,
 - **vector instructions**,
 - **single-instruction multiple-threads (SIMT)**, or
 - **tensor instructions**,
- where a single instruction can be used to initiate multiple operations.

Key Metrics and Design Objectives

- The **number of PEs** and the **peak throughput of a single PE** only indicate the theoretical maximum throughput (i.e., peak performance) when all PEs are performing computation (100% utilization).
- In reality, the achievable throughput depends on the actual utilization of those PEs, which is affected by several factors as follows:

$$\text{utilization of PEs} = \frac{\text{number of active PEs}}{\text{number of PEs}} \times \text{utilization of active PEs.}$$

- The first term reflects the **ability to distribute the workload to PEs**, while the second term reflects **how efficiently those active PEs are processing the workload**.
- The **number of active PEs** is the number of PEs that receive work; therefore, it is desirable to distribute the workload to as many PEs as possible.
- The ability to distribute the workload is determined by the flexibility of the architecture, for instance the on-chip network, to support the different layer shapes in the DNN model.

Key Metrics and Design Objectives

- Within the constraints of the on-chip network, the number of active PEs is also determined by the specific **allocation of work to PEs** by the mapping process.
- The **mapping process** involves the **placement** and **scheduling** in space and time of every MAC operation (including the delivery of the appropriate operands) onto the PEs. The mapper can be thought of as a compiler for the DNN hardware.
- The **utilization of the active PEs** is largely dictated by the timely delivery of work to the PEs such that the active PEs do not become idle while waiting for the data to arrive.
- This can be affected by the **bandwidth** and **latency** of the (on-chip and off-chip) **memory** and **network (interconnections)**.
- The **bandwidth requirements** can be affected by the **amount of data reuse** available in the DNN model and the amount of data reuse that can be exploited by the memory hierarchy and dataflow.

Key Metrics and Design Objectives

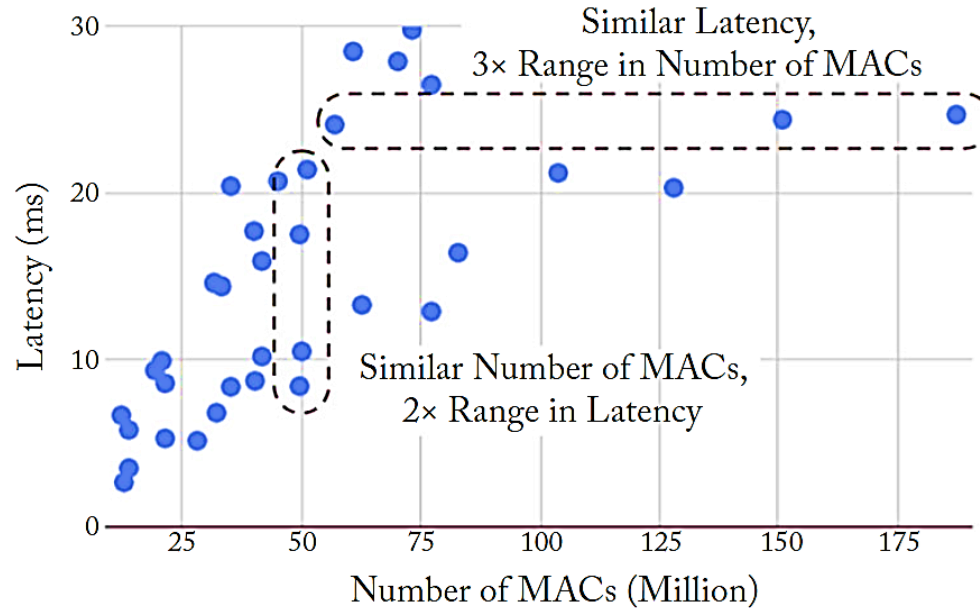
- The dataflow determines the order of operations and where **data is stored and reused**.
- The amount of **data reuse** can also be increased using a **larger batch size**, which is one of the reasons why increasing batch size can increase throughput.
- The **utilization of the active PEs** can also be affected by the **imbalance of work allocated** across PEs, which can occur when **exploiting sparsity** (i.e., avoiding unnecessary work associated with multiplications by zero); PEs with less work become idle and thus have lower utilization.
- There is also an interplay between the **number of PEs** and the **utilization of PEs**. For instance, one way to reduce the likelihood that a PE needs to wait for data is to store some data locally near or within the PE.
- However, this **requires increasing** the chip area allocated to **on-chip storage**, which, given a fixed chip area, would reduce the number of PEs. Therefore, a key design consideration is the **area allocation** between compute (which increases the number of PEs) versus on-chip storage (which increases the utilization of PEs).

Key Metrics and Design Objectives

- While the **number of operations per inference** depends on the DNN model, the **operations per second** depends on both the DNN model and the hardware.
- A deeper consideration of the **operations per second**, is that all operations are not created equal and therefore cycles per operation may not be a constant.
- For **example**, if we consider the fact that anything multiplied by zero is zero, some MAC operations are ineffectual (i.e., they do not change the accumulated value).
- The number of **ineffectual operations** is a function of both the DNN model and the input data. These ineffectual MAC operations can require fewer cycles or no cycles at all.
- Conversely, we only need to process effectual (or non-zero) MAC operations, where both inputs are non-zero; this is referred to as **exploiting sparsity**.

Key Metrics and Design Objectives

- **Figure 7:** The number of MAC operations in various DNN models versus latency measured on Pixel phone. Clearly, the number of MAC operations is not a good predictor of latency.



Key Metrics and Design Objectives

- **Energy efficiency** is used to indicate the amount of data that can be processed or the number of executions of a task that can be completed for a given unit of energy.
- High energy efficiency is important when processing DNNs at the edge in **embedded devices** with limited battery capacity (e.g., smartphones, smart sensors, robots, and wearables).
- **Edge processing** may be preferred over the cloud for certain applications due to latency, privacy, or communication bandwidth limitations. Energy efficiency is often generically reported as the number of operations per joule.
- In the case of inference, energy efficiency is reported as **inferences per joule** and energy consumption is reported as **joules per inference**.

Key Metrics and Design Objectives

- **Power consumption** is used to indicate the amount of energy consumed per unit time.
- Increased power consumption results in increased heat dissipation; accordingly, the maximum power consumption is dictated by a design criterion typically called the **thermal design power** (TDP), which is the power that the **cooling system** is designed to dissipate.
- Power consumption is important when processing DNNs in the cloud as data centers have stringent power ceilings due to cooling costs; similarly, handheld and wearable devices also have tight power constraints since the user is often quite sensitive to heat and the form factor of the device limits the cooling mechanisms (e.g., no fans).
- Power consumption is typically reported in **watts or joules per second**.

Key Metrics and Design Objectives

- Power consumption in conjunction with **energy efficiency** limits the **throughput** as follows:

$$\frac{\text{inferences}}{\text{second}} \leq \text{Max} \left(\frac{\text{joules}}{\text{second}} \right) \times \frac{\text{inferences}}{\text{joule}}.$$

- Therefore, if we can improve **energy efficiency** by increasing the number of **inferences per joule**, we can increase the number of inferences per second and thus **throughput** of the system.
- There are several factors that affect the energy efficiency. The **number of inferences per joule** can be decomposed into

$$\frac{\text{inferences}}{\text{joule}} = \frac{\text{operations}}{\text{joule}} \times \frac{1}{\frac{\text{operations}}{\text{inference}}},$$

Key Metrics and Design Objectives

- where the number of operations per joule is dictated by both the hardware and DNN model, while the number of operations per inference is dictated by the DNN model.
- There are various design considerations for the hardware that will affect the energy per operation (i.e., joules per operation).
- The energy per operation can be broken down into the energy required to move the input and output data, and the energy required to perform the MAC computation

$$\text{Energy}_{total} = \text{Energy}_{data} + \text{Energy}_{MAC}.$$

- For each component the joules per operation is computed as

$$\frac{\text{joules}}{\text{operation}} = \alpha \times C \times V_{DD}^2,$$

- where C is the total switching capacitance, V_{DD} is the supply voltage, and α is the switching activity, which indicates how often the capacitance is charged.

Key Metrics and Design Objectives

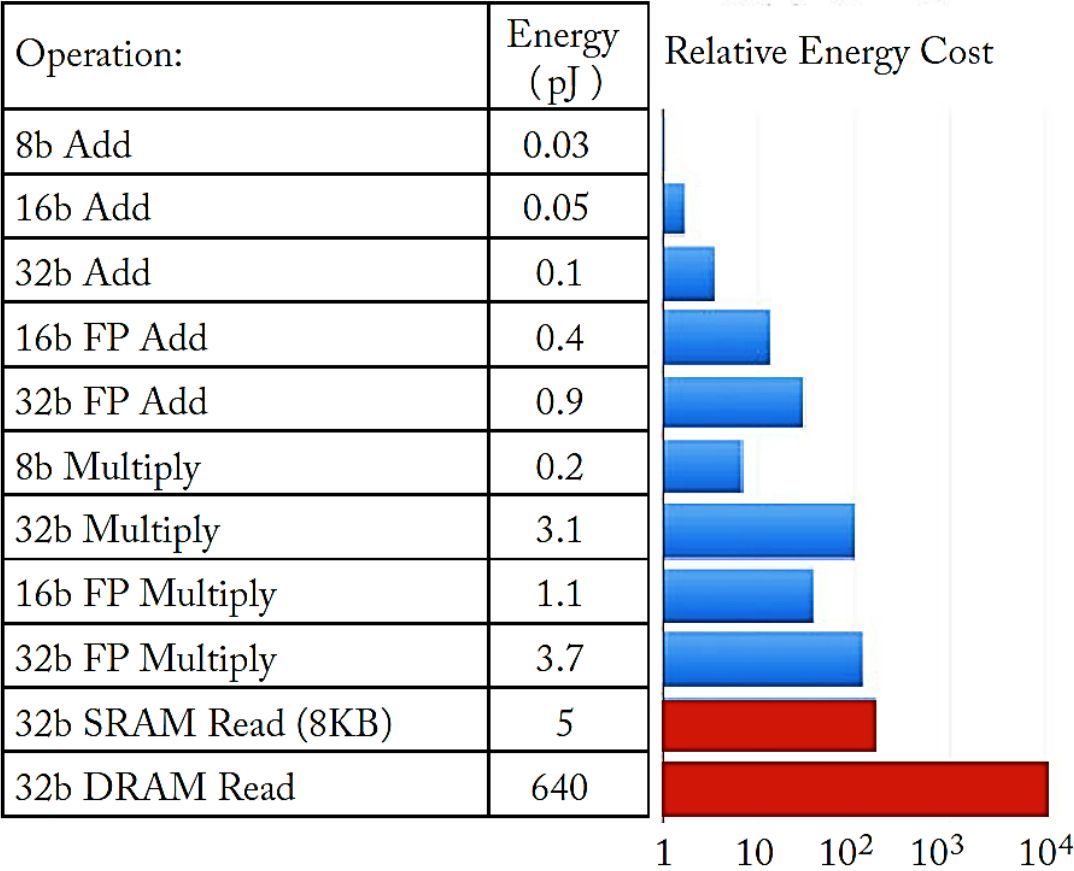


Figure 8: The energy consumption for various arithmetic operations and memory accesses in a 45 nm process.

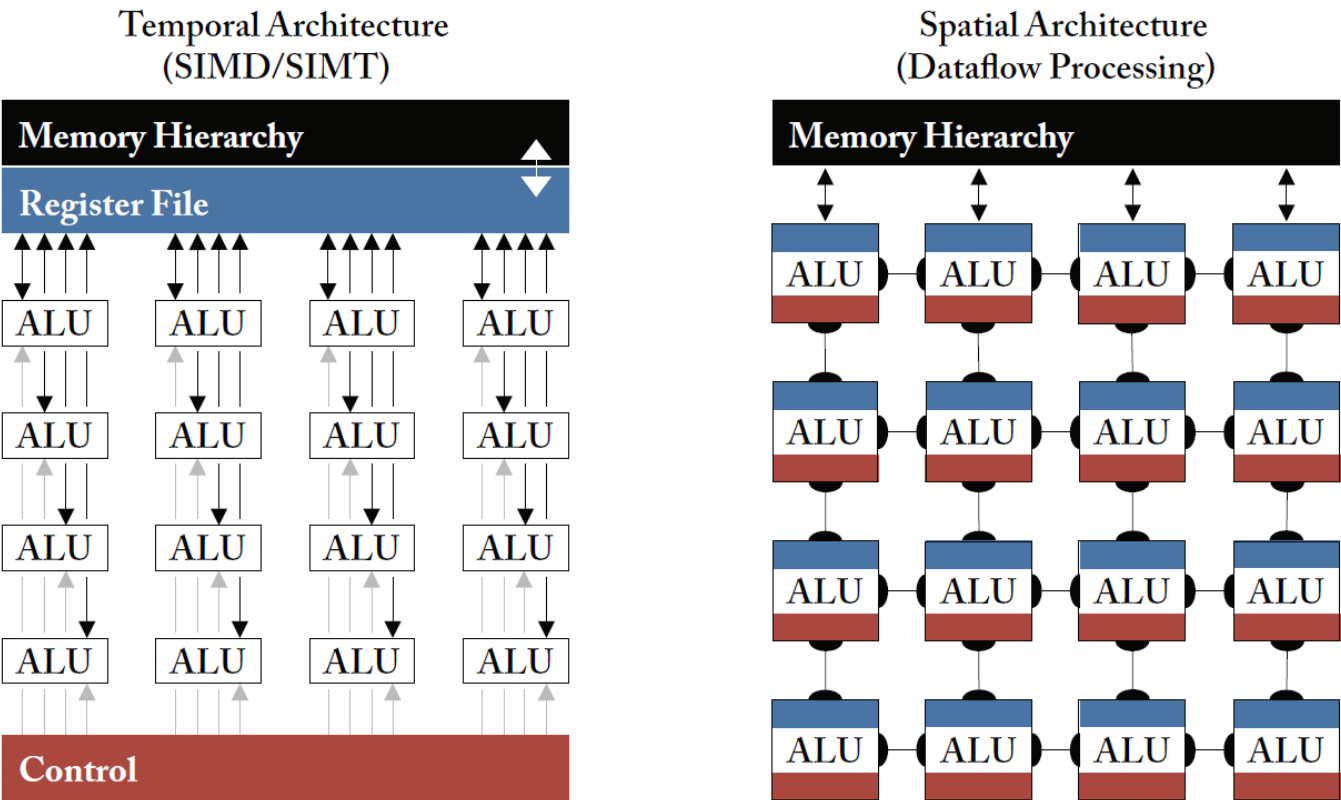
- The relative energy cost (computed relative to the 8b add) is shown on a log scale.
- The energy consumption of data movement (red) is significantly higher than arithmetic operations (blue).

Key Metrics and Design Objectives

- There are also other costs as:
- **Hardware cost** is used to indicate the monetary cost to build a system. This is important from both an industry and a research perspective as it dictates whether a system is financially viable. From an industry perspective, the cost constraints are related to volume and market; for instance, embedded processors have a much more stringent cost limitations than processors in the cloud.
- **Flexibility**, which refers to the range of DNN models that can be supported on the DNN processor and the ability of the software environment (e.g., the mapper) to maximally exploit the capabilities of the hardware for any desired DNN model. Given the fast-moving pace of DNN research and deployment, it is increasingly important that DNN processors support a wide range of DNN models and tasks.
- **Scalability**, which refers to how well a design can be scaled up to achieve higher performance (i.e., latency and throughput) and energy efficiency when increasing the number of resources (e.g., the number of PEs and on-chip storage). This evaluation is done under the assumption that the system does not have to be significantly redesigned (e.g., the design only needs to be replicated) since major design changes can be expensive in terms of time and cost. Ideally, a scalable design can be used for low-cost embedded devices and high-performance devices in the cloud simply by scaling up the resources.

Key Metrics and Design Objectives

Figure 9: Highly parallel compute paradigms.



Key Metrics and Design Objectives

- The challenges of designing specialized DNN hardware involve designing a **flexible architecture** and then finding the best ways to configure the architecture to get **optimal hardware performance** and **energy efficiency** for different DNN layers.
- These two aspects are tightly correlated, since the chance of finding the optimal configuration depends on the flexibility of the hardware, while higher flexibility often implies a loss of efficiency since additional hardware is required.
- Therefore, it often takes an **iterative process** to distill down to the best design.
- This is in contrast to the approach taken in the approaches, where the hardware architecture is already fixed, and the focus is on adapting the computation to fit in the compute paradigm of the hardware.

Key Metrics and Design Objectives

- Given a particular DNN model, it is necessary to be able to configure the hardware to minimize the overall energy consumption while maintaining high performance.
- This process involves finding an **optimal mapping**, where a mapping defines:
 - 1) the execution order of the MAC operations, both temporally (i.e., serial order on the same PE) and spatially (i.e., across many parallel PEs);
 - 2) how to tile and move data across the different levels of the memory hierarchy to carry out the computation in accord with that execution order.
- For a given DNN layer, there often exists a large number of possible mappings.
- As a result, it is very crucial to be able to find the best mappings for the desired metrics.

Key Metrics and Design Objectives

- Given the sheer number of possible **spatio-temporal orderings of MAC operations** for a DNN layer, it is generally very unlikely for a hardware architecture to support the execution of all these orderings.
- Therefore, practical hardware will support a more restricted set of orderings, which consequently reduces the number of legal mappings.
- Therefore, a very critical design consideration for DNN hardware is to select a subset of mappings to support. In other words, the hardware architecture has to set up rules that narrow down the range of considered mappings in the optimization.
- An important set of rules that determine the subset of supported mappings is called a dataflow.
- Dataflow is one of the most important attributes that define the architecture of a DNN hardware, as the subset of supported mappings directly impact the quality of the optimization.

Key Metrics and Design Objectives

- An additional design consideration is that the **shape** and **size** of a DNN layer can vary dramatically across different DNNs and also across different layers within a DNN.
- Furthermore, the **number of layers** that are found in emerging DNNs continues to grow. Therefore, given this rapidly growing and changing field, it is increasingly important to design DNN hardware that is **sufficiently flexible** and **scalable** to address these varying needs.
- Specifically, the hardware should avoid making assumptions about the layer shape and size; for instance, since DNNs have tended to grow in size, one cannot assume the entire model can always be stored on-chip.
- The degree of flexibility needed in the hardware to efficiently support a wide variety of DNNs has become one of the main challenges in the design of DNN accelerators.
- To summarize, the design and use of specialized DNN hardware involves multiple steps.

Design Steps

- At **design time**, an architecture is specified with a set of attributes. These attributes include:
- 1) the dataflow or dataflows supported;
 - 2) the number of PEs and the number of multipliers and adders per PE;5
 - 3) the memory hierarchy, including the number of storage levels and the storage capacity at each level; and
 - 4) the allowed patterns of data delivery for the NoC within the memory hierarchy and between the memory and PEs. Note, these attributes set certain limitations on the legal mappings of an architecture.
- For **example**, the amount of storage capacity at each level of the memory hierarchy impacts how much data reuse can be exploited in the hardware, which limits the set of supported mappings.

Design Steps

- **At mapping time**, given a DNN model, a mapping that optimizes the desired operational metrics is selected from among all the mappings supported by the accelerator.
- **At configuration time**, a configuration derived from the selected mapping is loaded into the accelerator.
- **At runtime**, the desired DNN input data is loaded and processed according to the loaded configuration. However, depending on the capabilities of the accelerator (e.g., how many layers can it process per configuration) it might need to iterate between the configuration step and the run step multiple times.